

实验一：目标检测与识别实验报告

机器人学集成小组项目

中国海洋大学计算机科学与技术学院

个人 GitHub: <https://github.com/9161942-boop/robotics-experiment-1>

Abstract—桌面场景中的目标检测看似简单，但物体尺寸、拍摄角度、遮挡和背景变化会直接影响识别结果。本实验以 **mouse**、**laptop**、**cup** 和 **phone** 四类桌面物体为对象，按照“数据采集—人工标注—格式转换—模型训练—独立测试—边缘部署”的完整流程完成目标检测实验。训练过程只使用本实验自行采集和人工标注的数据，从 **yolov8n.yaml** 随机初始化开始，未加载 **COCO** 或其他外部预训练权重。

数据集共包含 **595** 张图像，按来源时间块划分为 **421** 张训练图像、**87** 张验证图像和 **87** 张测试图像。最终四类模型在验证集上的精确率、召回率、**mAP@0.5** 和 **mAP@0.5:0.95** 分别为 **0.658**、**0.748**、**0.766** 和 **0.701**。系统在 **Jetson Orin** 上实时显示类别、边界框、置信度、物体数量和帧率，并通过 **ROS2** 发布 **/detections**。在完全独立、每类 **20** 张的 **80** 张实物照片上，系统通过 **72** 张，图片级准确率为 **90%**；**Jetson** 演示视频显示约 **15 FPS**。报告进一步说明标注质量控制、训练参数、评价口径、典型错误和材料归档方式。

Index Terms—目标检测，**YOLOv8**，**Jetson**，**ROS2**，独立实物测试，实验报告

I. 实验目的与任务要求

A. 实验目标

本实验的目的是掌握从数据准备到边缘设备运行的目标检测完整流程。实验要求选择不少于两类桌面物体，自行采集并标注数据，训练目标检测模型，在 **Jetson** 上实时显示检测框、类别和置信度，并通过 **ROS2** 发布识别结果。系统输入为 **USB** 摄像头的连续图像，核心模型为四类 **YOLOv8n**；完整链路为：摄像头采集 → **YOLOv8** 检测 → **Jetson** 实时显示 → **ROS2** 发布。

B. 实验对象与验收口径

本次实验选择 **mouse**、**laptop**、**cup** 和 **phone** 四类常见桌面物体。四类目标分别包含小尺寸目标、细长轮廓、反光表面和外观变化较大的电子设备，能够检验模型在常见桌面场景下的识别效果。训练集和验证集用于学习与模型选择，**test** 集用于训练结束后的带标注诊断，另外使用完全未参与训练的 **80** 张独立实物照片进行面向现场验收的图片级检查。这样既能用 **Precision**、**Recall** 和 **mAP** 检查检测框质量，也能用实物照片检查模型是否能识别真实物体。

实验结果按照课程要求分别检查四个方面：类别数量是否不少于两类，独立物体正确识别率是否达到 **80%**，**Jetson** 实时速度是否达到 **5 FPS**，以及结果和典型错误是否保存。独立照片的“通过”定义为目标类别正确且至少有一个检测框覆盖目标主体；该统计没有额外人工框，因此只作为实物验收指标，不替代带标注数据上的 **IoU** 和 **mAP**。

验收结果 独立实物照片测试 72/80 (90%)，高于 80% 门槛；**Jetson** 视频画面显示约 15 FPS，高于 5 FPS 门槛；数据集、模型、程序、视频、运行说明和错误案例均已归档。

II. 实验原理与实验流程

A. 实验原理

YOLOv8 将输入图像经过一次网络前向传播，直接预测目标类别和边界框，再通过非极大值抑制 (**NMS**) 去除重复框。对本实验而言，模型输出不仅要回答“画面中有没有目标”，还要回答“目标属于哪一类”和“目标在图像中的位置在哪里”。因此，标注框的位置、大小和类别编号都会影响训练结果，标签错位或漏标会直接造成分类混淆和定位误差。

训练时，模型根据人工标注框计算定位、分类和分布焦点损失，通过反向传播更新网络参数；验证阶段通过 **IoU** 匹配预测框与标注框，统计精确率、召回率和 **mAP**。推理阶段不再更新参数，只执行图像预处理、网络前向传播和 **NMS**。部署时，摄像头图像先进入检测节点，模型输出再同时送往显示模块和 **ROS2** 发布模块。这个过程将离线训练得到的权重转化为现场可以观察和复用的检测结果。

B. 实验流程

实验遵循“采集—筛选—标注—转换—训练—验证—独立测试—部署—验收—归档”的顺序。各阶段均保留对应文件：原始图片和 **ISAT JSON** 记录采集与标注依据，**YOLO** 目录保存训练输入，训练目录保存曲线、参数和权重，独立测试目录保存逐图结果与失败样本，**Jetson** 目录保存部署脚本和演示视频，最终通过 **GitHub** 的分阶段 **commit** 固化全过程。

流程中包含两条相互补充的验证线。第一条是带人工框的离线评估，用于检查预测框和真实标注之间的 **IoU**、**Precision**、**Recall** 与 **mAP**；第二条是完全独立

的实物照片测试，用于检查模型在新机型、新角度和新背景下是否仍能给出正确类别。前者回答“检测框是否准确”，后者回答“实际物体是否能被识别”，两者的统计口径不同，不能相互替代。



图 1. 从原始视频到可验收系统的完整实验流程

C. 具体操作步骤

- 1) 从桌面物体视频中抽帧，去除过暗、严重模糊和目标不完整的画面，再保留不同角度、距离和背景的本。
- 2) 使用 ISAT 为每个可见目标绘制紧致边界框，统一使用四个英文类别名，并逐张检查框是否越界、漏标或错标。
- 3) 将 ISAT JSON 转换为 YOLO detection 格式，即每行一个目标的类别编号与归一化中心坐标、宽度和高度。
- 4) 按来源时间块建立 train/val/test 划分，避免相邻视频帧同时出现在训练和评估集合中，并检查图像与标签一一对应。
- 5) 使用 yolov8n.yaml 随机初始化网络，在自建四类数据集上训练 100 个 epoch；训练过程中不加载 COCO 或其他外部 checkpoint，并记录参数、随机种子、曲线、混淆矩阵和权重。
- 6) 在验证集和 test 诊断集上计算 Precision、Recall、mAP，并结合可视化结果定位漏检、误检和类别混淆。
- 7) 使用完全未参与训练的 80 张独立实物照片进行图片级筛查，按“目标是否被正确识别”统计每类及总体通过率。
- 8) 将四类权重部署到 Jetson Orin，运行摄像头程序，检查检测框、类别、置信度、物体数量和滑动平均 FPS。
- 9) 通过 ROS2 节点把图像检测结果封装为 Detection2DArray 并发布到 /detections，最后整理报告、材料清单和 GitHub 记录。

III. 实验环境与数据准备

4. 实验设备与软件

表 I. 实验环境与数据配置

训练平台	Windows 11 + RTX 4060 Laptop GPU
部署平台	Jetson Orin, Ubuntu 20.04
框架与版本	YOLOv8n, Ultralytics 8.4.22
ROS2	Humble, 消息为 Detection2DArray
输入图像	640×480 USB 摄像头
类别与规模	mouse / laptop / cup / phone; 图像总数 595 (421/87/87)

训练机使用 Windows 11 和 RTX 4060 Laptop GPU，部署机为 Jetson Orin，系统为 Ubuntu 20.04。训练和部署均采用 Ultralytics YOLOv8n，ROS2 接口使用 Humble 版本的标准消息类型。摄像头输入按 640×480 采集；Jetson 推理时将图像缩放到 320，以兼顾处理速度和识别效果，输出视频仍保留原始画面比例。

训练目录和部署目录承担不同职责。训练目录保存数据配置、参数文件、损失曲线、混淆矩阵和最佳权重，便于复现模型选择过程；部署目录保存推理脚本、权重上传脚本和 ROS2 快速启动说明，便于在 Jetson 上按固定命令启动。报告中使用的结果图均来自这些目录中的实际文件，而不是重新绘制的示意图。

训练目录和部署目录承担不同职责。训练目录保存数据配置、参数文件、损失曲线、混淆矩阵和最佳权重，便于复现模型选择过程；部署目录保存推理脚本、权重上传脚本和 ROS2 快速启动说明，便于在 Jetson 上按固定命令启动。报告中使用的结果图均来自这些目录中的实际文件，而不是重新绘制的示意图。

B. 数据采集与划分

数据来自桌面物体视频抽帧，类别为 mouse、laptop、cup 和 phone，共 595 张图像。训练集、验证集和测试集分别为 421、87 和 87 张。数据划分依据整理得到的 19 个实物或视频来源，并在每个来源内部按时间块分配到不同集合，从源头降低相邻视频帧造成的数据泄漏风险。训练只使用 train/val，test 集保留到训练结束后用于诊断，使最终模型选择与测试集保持独立。

划分时没有简单地按文件名随机抽取相邻帧，而是先识别视频来源，再在来源内部按时间块分配。这样可以避免同一个物体在几乎相同姿态下同时出现在训练集和验证集，减少由于连续帧相似造成的虚高指标。独立实物照片不属于上述 595 张训练数据，且每个类别固定 20 张，因此可以进一步观察模型对未见外观和摆放方式的适应能力。

C. 标注与格式转换

每张原始图像先在 ISAT 中人工绘制边界框并保存 JSON 标注，再由转换脚本写入 YOLO 标签。转换后的每一行遵循“class_id x_center y_center width height”格式，坐标相对于图像宽高归一化到 $[0, 1]$ 。类别编号由 data.yaml 统一管理，以避免训练和推理阶段出现名称错位。转换完成后检查图像数量、标签文件数量、类别编号范围和空标签情况，并将通过检查的目录作为训练输入。图 2 给出四类样本及其人工边界框。

标注检查分为三个层次：首先检查框是否覆盖目标主体且没有明显越界；其次检查类别编号和文件名是否一一对应；最后检查转换后的标签能否被训练程序正常读取。对于反光的 phone、细长的 laptop 边缘和尺寸较小的 mouse，边界框尽量贴合可见轮廓；对于被遮挡的目标，只标注能够确认属于该目标的可见区域。这样的检查虽然不能消除所有主观误差，但能避免格式错误成为训练失败的主要原因。

标注检查分为三个层次：首先检查框是否覆盖目标主体且没有明显越界；其次检查类别编号和文件名是否一一对应；最后检查转换后的标签能否被训练程序正常读取。对于反光的 phone、细长的 laptop 边缘和尺寸较小的 mouse，边界框尽量贴合可见轮廓；对于被遮挡的目标，只标注能够确认属于该目标的可见区域。这样的检查虽然不能消除所有主观误差，但能避免格式错误成为训练失败的主要原因。

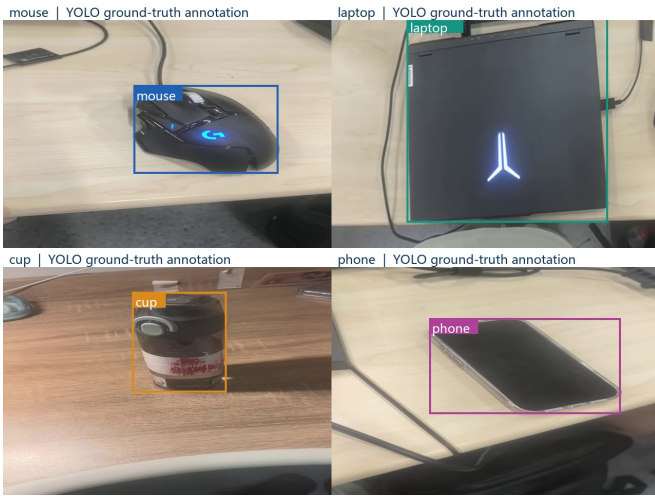


图 2. 数据集样本与人工标注框示例

IV. 实验过程与结果分析

A. 训练过程与控制条件

本实验的训练链路从自定义数据和随机初始化网络开始，未使用 COCO 或其他外部预训练模型。第一阶段直接调用 yolov8n.yaml，在 dataset/yolo/data.yaml 指定的四类数据上训练 100 个 epoch，输入尺寸为

640，batch size 为 16，随机种子为 42，workers 为 0，并启用自动优化器和 AMP 混合精度。训练阶段采用 mosaic、随机翻转、缩放和 RandAugment 等增强，最后 10 个 epoch 关闭 mosaic 以稳定收敛，得到本实验内部生成的 checkpoint。第二阶段以该自训练 checkpoint 作为内部起点，根据验收任务将检测头整理为 mouse、laptop、cup、phone 四类输出，并在同一自建数据集上继续训练，形成最终的四类权重 weights/best.pt。因此，最终模型仍属于基于自建标注数据的自行训练结果，整个过程没有引入外部预训练参数。验证阶段使用 Ultralytics 默认 NMS，离线可视化置信度阈值设为 0.25；Jetson 实时演示将阈值提高到 0.35，以减少低置信度框对画面的干扰。训练配置、曲线、混淆矩阵和权重均保存在仓库中。

第一阶段随机初始化模型在 test 诊断集上的标准 Ultralytics 评估为 $mAP@0.5=0.993$ 、 $mAP@0.5:0.95=0.960$ ，用于确认数据转换和训练管线能够正常工作。报告后续的验证集表格、独立实物照片和 Jetson 演示结果，均对应第二阶段形成的最终四类权重。

两阶段训练保持了相同的数据配置、输入尺寸和随机种子，变化集中在模型输出头和第二阶段权重初始化方式。这样做的目的不是把两个模型的指标混为一谈，而是先验证“自建数据能否从零训练”，再得到满足课程四类输出要求的最终模型。训练日志、args.yaml、曲线和权重文件共同构成训练过程证据，报告中的验收结论只引用最终四类模型的结果。

B. 指标与结果分析

精确率 (Precision) 衡量预测框中真正例的比例，召回率 (Recall) 衡量标注目标被找回的比例； $mAP@0.5$ 和 $mAP@0.5:0.95$ 分别在单一 IoU 阈值和多个 IoU 阈值下汇总各类别平均精度。验证集指标用于观察模型泛化并选择最佳权重，test 诊断结果用于检查未参与训练的帧级表现，二者均依赖人工边界框。独立照片测试没有额外人工框，因此采用图片级“是否正确识别”统计，作为面向实物验收的补充指标。

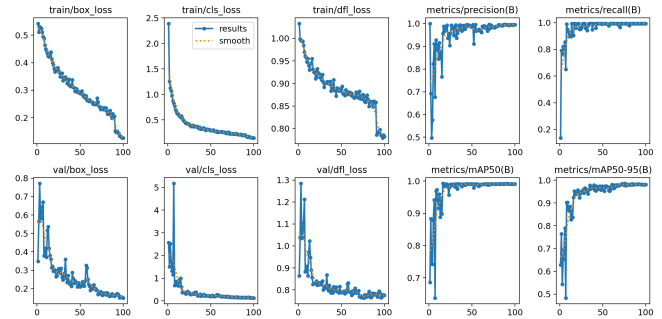


图 3. 训练过程指标曲线（对应仓库 results/training/results.png）

表 II. 验证集与诊断集结果

数据/指标	P	R	mAP@50	mAP@50:95
dataset/yolo 验证集	0.658	0.748	0.766	0.701

从类别结果看，最终模型在验证集上的 mAP@0.5:0.95 分别为 mouse 0.723、laptop 0.877、cup 0.428 和 phone 0.776。laptop 的轮廓和面积通常较大，定位相对稳定；cup 的外观受杯口、把手、透明或反光材质影响更明显，因此是四类中最需要补充困难样本的类别。这个类别差异与独立照片测试中 cup 通过率最低的现象一致，说明错误并非只来自某一张图片，而是与目标外观和背景对比度有关。

训练曲线用于观察损失是否下降以及验证指标是否趋于稳定，混淆矩阵用于检查类别之间的系统性混淆。两类图表分别回答“模型是否学会”和“模型容易把什么认错”两个问题；因此本实验没有只报告一个最终数字，而是同时保留曲线、矩阵、逐图结果和失败样本，方便在验收时解释结果来源。

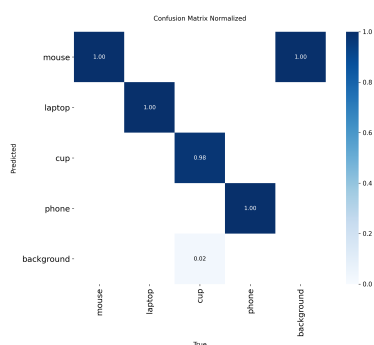


图 4. 四类目标的归一化混淆矩阵

独立验收集共 80 张照片，每个类别 20 张，按“该照片中目标是否被正确识别”统计：cup 17/20、laptop 19/20、mouse 18/20、phone 18/20，总计 72/80 (90%)。未通过样本为 04_cup.jpg、12_cup.jpg、13_cup.jpg、21_laptop.jpg、42_mouse.jpg、46_mouse.jpg、72_phone.jpg、80_phone.jpg。

80 张照片来自训练数据之外的独立实物集合，每类固定 20 张，覆盖不同机身、摆放方向、距离和背景。判定规则为：目标类别预测正确，且至少有一个检测框覆盖目标主体，则该图片通过；漏检、类别错误或仅检测到背景均记为未通过。该规则与课程“测试 20 个物体、正确率不低于 80%”的现场检查方式一致，同时给出四类分项结果以定位薄弱类别。cup 的通过率为 85%，laptop 为 95%，mouse 和 phone 均为 90%。

从分项结果看，四类均超过课程要求的 80% 单类门槛，总体通过率为 90%。laptop 的 19/20 说明大目标和清晰轮廓较容易被模型定位；cup 的 17/20 是当

前最弱项，主要受杯体边缘、把手和桌面颜色接近影响；mouse 的两张失败样本与小尺寸和遮挡有关；phone 的两张失败样本则集中在新机型外观、低对比度和摆放方向变化。这里的分析只针对图片级通过与失败，不把它解释为带 IoU 标注的检测精度。

表 III. 独立实物照片的类别级通过统计

类别	测试数	通过数	通过率
cup	20	17	85%
laptop	20	19	95%
mouse	20	18	90%
phone	20	18	90%
总计	80	72	90%

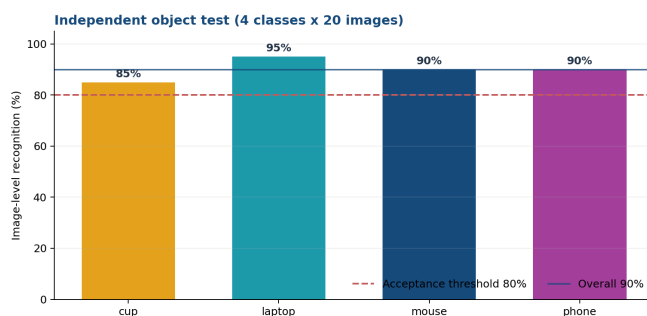


图 5. 80 张独立实物照片的分类型通过率与 80% 验收门槛

V. 系统部署与功能验证

A. Jetson 实时检测

Jetson 演示视频分辨率为 640×480。程序从摄像头读取连续帧，在 GPU 上完成 YOLOv8n 推理和 NMS，再将检测框、类别、置信度、物体数量和滑动平均 FPS 叠加到画面。原视频中的 FPS 约为 15.0，满足不低于 5 FPS 的验收要求；抽帧中可以同时看到 mouse、phone、laptop 和 cup 的检测框。部署脚本支持通过 SSH 上传代码并指定权重，现场只需调整摄像头设备号或实际图像话题即可复用。

部署时将推理尺寸设为 320，而摄像头采集和输出视频仍保持 640×480。该设置把计算量控制在 Jetson 可承受范围内，同时保留足够的画面信息用于现场观察。程序显示的 FPS 是连续处理过程中的滑动平均值，能够反映摄像头读取、推理、绘制和显示这一整条链路的实际速度，而不是只统计一次模型前向传播时间。

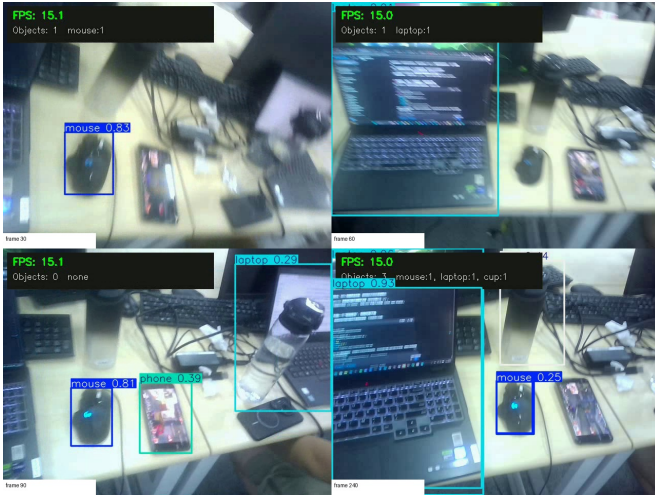


图 6. Jetson 演示视频抽帧（原视频：
results/videos/jetson_demo.mp4）

B. ROS2 结果发布

检测节点不创建摄像头驱动，而是订阅相机节点发布的 `sensor_msgs/Image`。节点对每帧执行与实时程序一致的预处理、推理和置信度筛选，并将检测结果封装为 `vision_msgs/Detection2DArray`：消息头记录时间戳，每个检测项包含类别名、置信度和像素坐标框。检测节点订阅 `/camera/image_raw`，推理后发布 `/detections`。程序、启动方式、参数和验证命令写入 `docs/ros2_quickstart.md`，现场验证命令为：

```
ros2 topic echo /detections --once
```

ROS2 发布模块与显示模块共享同一帧推理结果，因此画面中看到的框和消息中的框具有相同的类别与置信度。消息采用标准 `Detection2DArray`，后续节点可以继续订阅它完成计数、跟踪或机械臂动作，而不需要重新读取摄像头或重复运行模型。这使本实验的输出从“能在窗口里显示”扩展为“能被其他 ROS2 节点复用”。

VI. 问题分析与实验总结

A. 典型错误

错误案例已保存于 `results/independent_test_80/errors/` 和 `results/evaluation_final/errors/`。8 张未通过照片中，`cup` 失败 3 张，主要表现为目标边缘与桌面背景对比度不足；`laptop` 失败 1 张，与斜视角和屏幕反光有关；`mouse` 失败 2 张，与目标尺寸较小或被遮挡有关；`phone` 失败 2 张，与新机型外观、低对比度和摆放方向变化有关。总体来看，失败主要与小目标、运动模糊、遮挡、反光和低置信度有关，说明后续优化应优先补充困难样本和场景变化。

B. 复现方法与改进建议

在 Windows 训练机中，先运行 `python convert_isat_to_yolo.py` 生成或更新 `dataset/yolo/`，再运行 `python train_yolo.py --epochs 100 --batch 16 --device 0 --workers 0`。评估可运行 `python evaluate_yolo.py --device 0`；独立照片的逐图结果位于 `results/quality_4class_best_independent/`。最终权重的 SHA-256 完整值记录在仓库 README 中，报告中保留其首尾校验片段：

```
55341D55082158F799C0DA7ED813C959
0EE2AE043ED2AD837DBD0B47CB0E3FE4
```

可用该摘要确认下载文件未被替换。后续工作可补充困难样本和遮挡增强，提高小目标输入尺寸，并使用 TensorRT 进一步压缩延迟；这些方向属于改进计划，不改变本次验收结果。

本实验仍存在两个边界。第一，独立照片采用图片级通过规则，没有为 80 张照片重新绘制人工框，因此不能据此计算严格的 mAP；第二，Jetson 视频提供了 FPS 和画面证据，但没有单独记录每个 ROS2 消息的时间延迟。针对这些边界，报告分别保留了带标注验证集指标、独立照片逐图清单、失败案例和运行说明，避免用单一指标替代完整验收证据。

VII. 实验材料整理与提交记录

按照课程要求，报告、数据、代码、模型和结果均上传至个人 GitHub，并保留分阶段 commit 记录，未采用一次性全量提交。仓库链接为：<https://github.com/9161942-boop/robotics-experiment-1>。

提交目录按“输入—模型—程序—结果—说明—报告”组织。`dataset/images/` 保存原始图片和 ISAT 标注，`dataset/yolo/` 保存可直接训练的图像与标签，`weights/` 保存最终四类权重，`code/` 和 `docs/` 保存检测程序、ROS2 节点和启动说明，`results/` 保存验证图、独立测试统计、失败样本和 Jetson 视频，`report/` 保存 LaTeX 源文件与 PDF。这样的目录关系与实验流程一一对应，验收时可以从数据逐级追溯到最终画面。

表 IV. 代表性分阶段提交

阶段	Commit	内容
数据准备	5fcd148	鼠标样本与标注计划
数据准备	c2507b4	cup/laptop/phone 数据补充
训练管线	3e007d8	四类 YOLO 训练流程
独立测试	a93ffb5	独立图片筛查记录
Jetson 部署	14a710c	实时检测演示视频
验收整理	d887607	验收材料与 PDF 报告

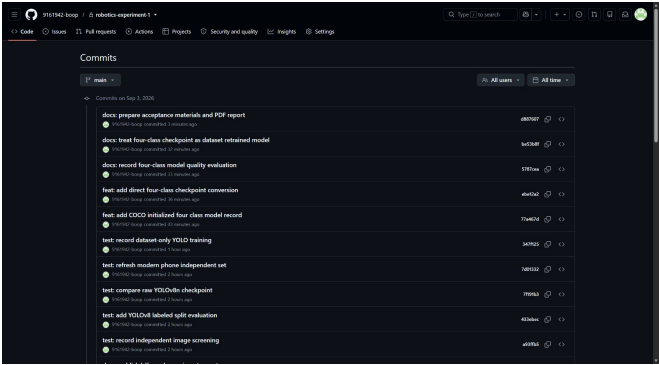


图 7. GitHub main 分支提交记录截图（页面访问时间：2026-09-03）

VIII. 验收结果与实验总结

提交材料包括：数据集 `dataset/images/`、`dataset/yolo/`；四类模型 `weights/best.pt`；检测与 ROS2 程序；Jetson 视频 `results/videos/jetson_demo.mp4`；运行说明 `README.md`、`docs/ros2_quickstart.md`；训练曲线、混淆矩阵、独立测试统计图、失败样本和报告源文件。材料目录同时保留原始数据、可直接训练的数据和最终验收结果，便于按“数据—模型—程序—视频—报告”的顺序检查。所有材料均通过 GitHub 分阶段 commit 留存。

表 V. 验收要求与材料证据对照

验收要求	对应证据	状态
同时识别不少于 2 类	四类模型（ <code>mouse</code> 、 <code>laptop</code> 、 <code>cup</code> 、 <code>phone</code> ）及 80 张独立照片测试记录	达成
20 个物体正确率不低于 80%	独立测试汇总：72/80，准确率 90%，并附类别统计图	达成
Jetson 实时速度不低于 5 FPS	<code>results/videos/jetson_demo.mp4</code> 中叠加显示约 15 FPS	达成
保存结果与典型错误案例	<code>results/independent_test_80/</code> 、 <code>results/evaluation_final/</code> 及错误样例目录	达成
ROS2 发布识别结果	节点发布 <code>/detections</code> ，启动与 <code>ros2 topic echo</code> 命令写入运行说明	已实现

IX. 现场验收操作清单

验收时可按照“先确认输入，再确认输出，最后确认证据”的顺序复核。该顺序与报告前面的实验流程一致，能够避免只展示视频而无法追溯模型和数据。

- 1) 检查 `dataset/yolo/data.yaml`、`weights/best.pt` 和运行环境是否存在，并确认模型类别为 `mouse`、`laptop`、`cup`、`phone` 四类。
- 2) 在训练机上运行评估脚本，查看验证集指标和混淆矩阵；需要说明验证集指标依赖人工框，独立照片指标采用图片级通过规则。
- 3) 将最终权重和检测程序上传 Jetson，连接 USB 摄像头后启动实时检测，现场观察检测框、类别、置信度、物体数量和 FPS 是否同步更新。
- 4) 依次展示不同角度的 `mouse`、`laptop`、`cup` 和 `phone`，确认系统可以在同一窗口中同时显示多个类别。

- 5) 打开 `results/videos/jetson_demo.mp4`，说明视频中的分辨率、检测信息和约 15 FPS 结果，并对照 5 FPS 的课程门槛。
- 6) 如需检查 ROS2 接口，启动相机话题和检测节点，再执行 `ros2 topic echo /detections --once`，确认消息类型和输出话题正确。
- 7) 最后打开独立测试汇总和错误样本目录，说明 80 张照片中 72 张通过，并指出 8 张未通过图片及其原因。

现场说明时应把“模型效果”和“材料证据”同时展示：前者由实时画面和独立实物测试体现，后者由数据集、标注文件、权重、代码、视频、运行说明和 GitHub commit 记录体现。这样既能回答系统能否运行，也能回答结果是否可复现、过程是否可追溯。

结论

通过本次实验，完成了从数据采集、标注、YOLOv8 训练到 Jetson 实时检测和 ROS2 结果发布的完整操作。四类独立实物照片通过率为 90%，Jetson 视频显示约 15 FPS，达到实验的主要验收指标。实验报告源文件、数据集、模型、程序、视频、运行说明和错误案例均已整理并上传 GitHub，后续可以据此复现实验过程。